ソフトウェア情報学
基礎ゼミナールB
第７回

青森大学ソフトウェア情報学部
橋本 恭能
haship@aomori-u.ac.jp

目次

- ON/OFF制御のライントレース（復習）
- 分かりやすいプログラムに変更していく
  - クラスのメンバー変数
  - クラスのコンストラクタ
  - クラスのメンバー関数
- 自分で関数（メソッド）を作る

ON/OFF制御のライントレース（復習）

- ON-OFF制御
  - 設定値を基準にして、一定した動作をON・OFFする制御方法
  - 利点：仕組みが単純
  - 欠点：設定値の前後で振動する、安定しない
- 簡単な仕組みでライントレース可能

クラス

- Javaのクラス

```
class クラス名 {

    メンバー変数の宣言

    コンストラクタ

    メンバー関数(メソッド)の定義
}
```

- メンバー変数の宣言

```
データ型　変数名;

クラス名　インスタンス名;
```

- コンストラクタ

```
クラス名(パラメータ){

    //クラスのインスタンスを生成した時に
    //自動で実行される
    //メンバー変数の初期化・初期設定を実行する

}
```

- メンバー関数の定義

```
データ型 関数名(パラメータ){

}

クラス名 関数名(パラメータ){

}
```

変更点①

```
import lejos.nxt.Button;
(以下省略)

public class LineTracer01 {

  private static final int THRESHOLD = 50;

  public static void main(String[] args) {

  // インスタンスを生成する
  NXTMotor leftMotor = new NXTMotor(MotorPort.C);
  NXTMotor rightMotor = new NXTMotor(MotorPort.B);
  TouchSensor touch = new TouchSensor(SensorPort.S1);
  LightSensor light = new LightSensor(SensorPort.S3);

  (以下略)

  }

}
```

クラス名
LineTracer01

点線部分を
メンバー変数に
変更

```
import lejos.nxt.Button;
(以下省略)

public class LineTracer01 {

  private static final int THRESHOLD = 50;

  // インスタンスを生成する
  NXTMotor leftMotor = new NXTMotor(MotorPort.C);
  NXTMotor rightMotor = new NXTMotor(MotorPort.B);
  TouchSensor touch = new TouchSensor(SensorPort.S1);
  LightSensor light = new LightSensor(SensorPort.S3);

  public static void main(String[] args) {

  (以下略)

  }

}
```

まず
場所の変更
②に続く

変更点②

```
import lejos.nxt.Button;
(以下省略)

public class LineTracer01 {

  private static final int THRESHOLD = 50;

  //メンバー変数の宣言
  private NXTMotor leftMotor;
  private NXTMotor rightMotor;
  private TouchSensor touch;
  private LightSensor light;

  // コンストラクタ
  public LineTracer01(){
    leftMotor  = new NXTMotor(MotorPort.C);
    rightMotor = new NXTMotor(MotorPort.B);
    touch      = new TouchSensor(SensorPort.S1);
    light      = new LightSensor(SensorPort.S3);
  }

  public static void main(String[] args) {

  (以下略)
```

変更

変更

追加

- メンバー変数
  - NXTMotorクラスのleftMotor
  - NXTMotorクラスのrightMotor
  - TouchSensorクラスのtouch
  - LightSensorクラスのlight
- コンストラクタ追加
  - leftMotorのコンストラクタを呼出す
  - rightMotorのコンストラクタを呼出す
  - touchのコンストラクタを呼出す
  - lightのコンストラクタを呼出す

変更点③

```
import lejos.nxt.Button;
(以下省略)

public class LineTracer01 {
(以下略)

  public static void main(String[] args) {

    //クラスLineTracer01のインスタンスrobotを生成
    LineTracer01 robot = new LineTracer01();

    //設定
    robot.leftMotor.setPower(50);
    robot.rightMotor.setPower(50);
    robot.leftMotor.stop();
    robot.rightMotor.stop();
    robot.light.setFloodlight(true);

    (以下略)
```

追加

変更

- 新規にクラスLineTracer01のインスタンス生成
  名前 robot

robotのleftMotorのパワーを50に設定する

(変更前) leftMotor.setPower(50);

↓

(変更後) robot.leftMotor.setPower(50);

他の操作も同様に変更

変更後のプログラムリスト１

```
import lejos.nxt.Button;
import lejos.nxt.LCD;
import lejos.nxt.LightSensor;
import lejos.nxt.MotorPort;
import lejos.nxt.NXTMotor;
import lejos.nxt.SensorPort;
import lejos.nxt.TouchSensor;
import lejos.util.Delay;

public class LineTracer01 {

        private static final int THRESHOLD = 50;    //各自しきい値を変更する

        private NXTMotor leftMotor;
        private NXTMotor rightMotor;
        private TouchSensor touch;
        private LightSensor light;

        public LineTracer01(){
                leftMotor = new NXTMotor(MotorPort.C);
                rightMotor = new NXTMotor(MotorPort.B);
                touch = new TouchSensor(SensorPort.S1);
                light = new LightSensor(SensorPort.S3);
        }
```

変更後のプログラムリスト２

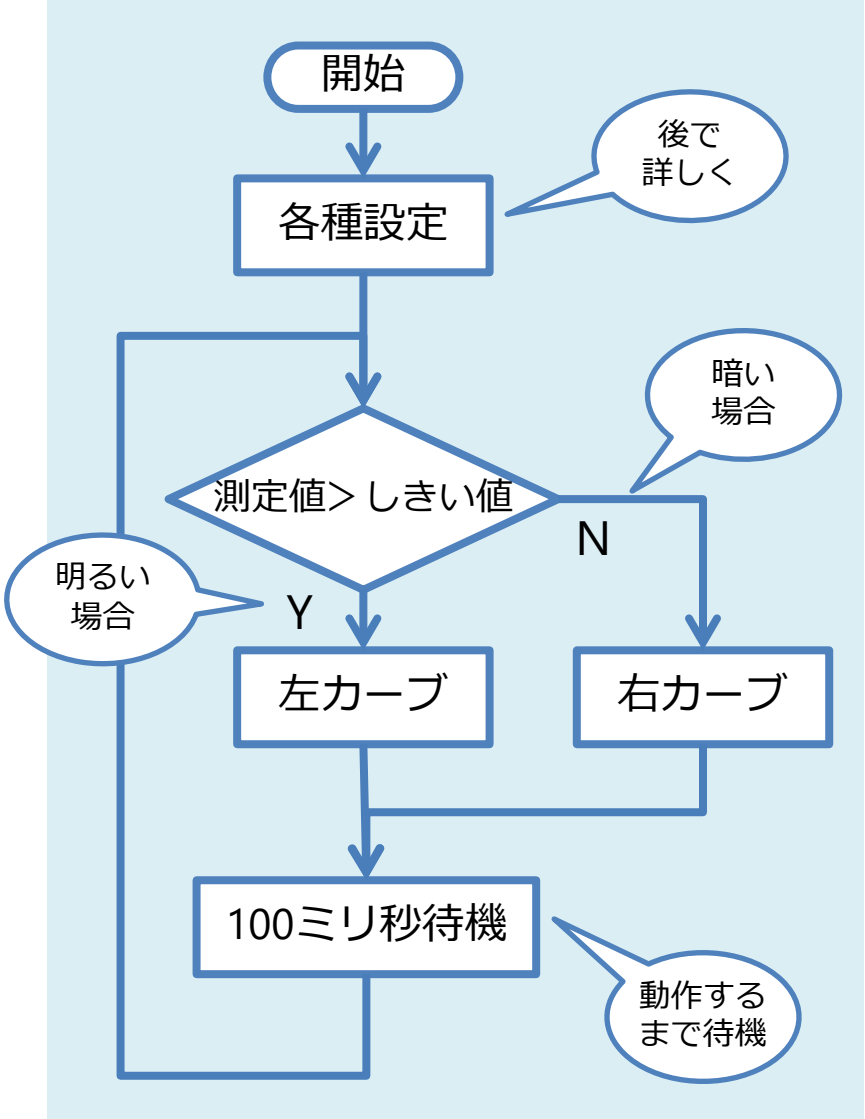
```
public static void main(String[] args) {

        //クラスインスタンス生成
        LineTracer01 robot = new LineTracer01();

        //設定
        robot.leftMotor.setPower(50);
        robot.rightMotor.setPower(50);
        robot.leftMotor.stop();
        robot.rightMotor.stop();
        robot.light.setFloodlight(true);

        //タッチセンサーを押したら走行開始する
        while(!robot.touch.isPressed()){

                //画面表示
                LCD.clear();
                LCD.drawInt(robot.light.readValue(), 0, 0);

                // 待機
                Delay.msDelay(100);
        }
```

変更後のプログラムリスト3

```
                    while (Button.ESCAPE.isUp()) {

                              int val = robot.light.readValue();

                              // 光センサーの値がしきい値より大きい場合
                              if (val > THRESHOLD) {
                                        robot.leftMotor.stop();
                                        robot.rightMotor.forward();

                              } else {
                                        robot.leftMotor.forward();
                                        robot.rightMotor.stop();

                              }
                              LCD.clear();
                              LCD.drawInt(val, 0, 0);

                              Delay.msDelay(100);
                    }
                    robot.leftMotor.stop();
                    robot.rightMotor.stop();
          }
}
```

仕組み・プログラムコードの対応

- フローチャートと
  プログラムコードで、
  どこが対応しているか
  わかりますか？
- 例えば・・・
  - 明るい場合・暗い場合
  - 左カーブ・右カーブ



分かりにくい処理を挙げてみる

- 暗い場合⇒光センサーが黒線の上にいる
  - 光センサーの測定値がしきい値より小さい場合
- 明るい場合⇒暗い以外の場合
- 左ターン
  - 左モーターを停止
  - 右モーターを順回転
- 右ターン
  - 左モーターを順回転
  - 右モーターを停止

- つまり説明で使っている表現とプログラムで書いていることは対応がつけにくいところがある
- プログラムを見てやっていることが分かるようにするともっと理解しやすいプログラムに

自分でメソッドを作る

- 光センサーが黒線の上にいるかどうか
  ⇒黒線の上にいる場合 true
  　それ以外（黒線の上にいない場合）false
- 光センサーの測定値 パラメータにする
- しきい値 THRESHOLD

```
public boolean isOnLine(int val){
  return val < THRESHOLD;
}
```

自分でメソッドを作る

- 左ターン turnLeft(int power)
  - 左モーターを停止　leftMotor.stop()
  - 右モーターを順回転rightMotor.forward()
  - パワーの設定も実行 setPower(power)

```
public void turnLeft(int power){
        leftMotor.setPower(power);
        rightMotor.setPower(power);
        leftMotor.stop();
        rightMotor.forward();
}
```

自分でメソッドを作る

- 右ターン turnRight(int power)
  - 左モーターを停止
  - 右モーターを順回転
  - パワーの設定も実行

```
public void turnLeft(int power){

        //自分で考えてみよう



}
```

自分でメソッドを作る

- 停止 stop()
  - 左モーターを停止
  - 右モーターを停止

```
public void stop(int power){

        //自分で考えてみよう



}
```

追加したメソッド

メソッド	機能
`public boolean isOnLine(int val)`	黒線上にいるかどうか パラメータ int val 光センサーの測定値 戻り値 true 黒線上にいる false 黒線上にいない
`public void turnLeft(int power)`	左ターンする パラメータ int power 左右モーターのパワー 戻り値　なし
`public void turnRight(int power)`	右ターンする パラメータ int power 左右モーターのパワー 戻り値　なし
`public void stop()`	停止する パラメータ　なし 戻り値　なし

再変更後のプログラムリスト１

```
import lejos.nxt.Button;
import lejos.nxt.LCD;
import lejos.nxt.LightSensor;
import lejos.nxt.MotorPort;
import lejos.nxt.NXTMotor;
import lejos.nxt.SensorPort;
import lejos.nxt.TouchSensor;
import lejos.util.Delay;

public class LineTracer01 {

        private static final int THRESHOLD = 50;    //各自しきい値を変更する

        private NXTMotor leftMotor;
        private NXTMotor rightMotor;
        private TouchSensor touch;
        private LightSensor light;

        public LineTracer01(){
                leftMotor = new NXTMotor(MotorPort.C);
                rightMotor = new NXTMotor(MotorPort.B);
                touch = new TouchSensor(SensorPort.S1);
                light = new LightSensor(SensorPort.S3);
        }
```

再変更後のプログラムリスト２

```
public static void main(String[] args) {

        //クラスインスタンス生成
        LineTracer01 robot = new LineTracer01();

        //設定
        robot.stop();
        robot.light.setFloodlight(true);

        //タッチセンサーを押したら走行開始する
        while(!robot.touch.isPressed()){

                //画面表示
                LCD.clear();
                LCD.drawInt(robot.light.readValue(), 0, 0);

                // 待機
                Delay.msDelay(100);
        }

        while (Button.ESCAPE.isUp()) {
```

再変更後のプログラムリスト３

```
        int val = robot.light.readValue();

        // robotが黒線の上にいる場合（暗い場合）
        if(robot.isOnLine(val)){

                //パワー50で左ターンする
                robot.turnLeft(50);

        } else {    //それ以外（明るい場合）

                //パワー50で右ターンする
                robot.turnRight(50);
        }

        //画面表示
        LCD.clear();
        LCD.drawInt(val, 0, 0);

        // 待機
        Delay.msDelay(100);
}
```

再変更後のプログラムリスト４

```
            //robotを停止
            robot.stop();
        }
        // ロボットが黒線上にいるかどうか
        public boolean isOnLine(int val){
            return val < THRESHOLD;
        }
        //左ターンする
        public void turnLeft(int power){
            leftMotor.setPower(power);
            rightMotor.setPower(power);
            leftMotor.stop();
            rightMotor.forward();
        }
        //右ターンする
        public void turnRight(int power){
            leftMotor.setPower(power);
            rightMotor.setPower(power);
            rightMotor.stop();
            leftMotor.forward();
        }
```

再変更後のプログラムリスト５

```
        //停止する
        public void stop(){

        }
}
```